

IST608 — Transaction Processing I

Practice Examination (100 Marks)

Source: *LECT TRANSACTION PROCESSING I* (Elmasri & Navathe, Ch. 17)

Coverage: Transactions, concurrency, recovery, schedules, serializability, SQL transaction support

Mark distribution

Section	Questions	Marks
A — Multiple Choice	20	20
B — Structured	6	30
C — Essay	5	50
Total		100

SECTION A — MULTIPLE CHOICE (20 marks)

Each question carries **1 mark**. Choose the **one** best answer.

1. A transaction is best defined as:

- (a) A physical disk block read operation
- (b) The smallest logical unit of database work treated as all-or-nothing
- (c) Any SQL query that returns more than one row
- (d) A user session lasting until logout

Answer: (b) — A transaction is a logical unit of processing that must complete entirely or not at all.

2. In the simple transaction model, `read_item(X)` ultimately copies data from:

- (a) Program variable to disk directly
- (b) Disk block buffer in main memory to the program variable
- (c) Cache to another cache line only
- (d) The log file to the program variable

Answer: (b) — The block containing X is brought into a buffer; then X is copied from the buffer to the program variable.

3. The **lost update** problem occurs when:

- (a) A transaction reads uncommitted data
- (b) Interleaved writes cause one transaction's update to be overwritten incorrectly

- (c) The system log is not force-written
- (d) A phantom row appears in a second read

Answer: (b) — Two transactions updating the same item without proper control can leave the database with a wrong final value.

4. A dirty read is the same as:

- (a) Incorrect summary problem
- (b) Temporary update problem
- (c) Disk head crash
- (d) Strict schedule violation only

Answer: (b) — Reading a value written by a transaction that may still abort (uncommitted).

5. Which is **not listed in the lecture as a cause of transaction failure?**

- (a) System crash
- (b) Integer overflow in the transaction
- (c) Deadlock detected by concurrency control
- (d) Successful COMMIT after force-write of log

Answer: (d) — COMMIT after proper logging indicates successful completion, not failure.

6. After a transaction reaches its **commit point, the next log entry is typically:**

- (a) `[start_transaction,T]`
- (b) `[abort,T]`
- (c) `[commit,T]`
- (d) `[read_item,T,X]` only

Answer: (c) — The commit point means effects are logged; then `[commit,T]` is written.

7. Force-writing the log before commit supports which ACID property most directly?

- (a) Atomicity only
- (b) Isolation only
- (c) Durability
- (d) Consistency only

Answer: (c) — Ensures committed changes can survive a crash via redo from the log.

8. Which log record type stores both **old_value and **new_value**?**

- (a) `[start_transaction,T]`
- (b) `[write_item,T,X,old_value,new_value]`
- (c) `[commit,T]` only
- (d) `[read_item,T,X]` always with old and new

Answer: (b) — Write records capture before-image and after-image for undo/redo.

9. Atomicity means:

- (a) Transactions run one at a time only
- (b) A transaction is performed in its entirety or not at all
- (c) All users see the same data
- (d) The database never uses a log

Answer: (b) — Standard ACID definition from the lecture.

10. A recoverable schedule requires that:

- (a) No transaction ever reads data
- (b) A transaction T does not commit until all transactions that wrote data T read have committed
- (c) Every schedule is serial
- (d) Blind writes are forbidden

Answer: (b) — Commit order must respect read-from dependencies.

11. Strict schedules are **stricter** than cascadeless schedules because strict schedules require:

- (a) No reads or writes on X until the last writer of X has **committed**
- (b) Only serial execution on one CPU
- (c) No log file
- (d) READ UNCOMMITTED isolation in SQL

Answer: (a) — Strict: neither read nor write X until the previous writer of X commits.

12. A schedule is conflict-serializable if and only if its precedence graph:

- (a) Has at least one cycle
- (b) Is acyclic
- (c) Has no nodes
- (d) Is complete (every pair of transactions connected both ways)

Answer: (b) — Algorithm 17.1: acyclic precedence graph $\Leftrightarrow$ conflict serializable.

13. Two operations conflict if they belong to different transactions, access the **same** item, and at least one is a:

- (a) Commit
- (b) Read or write (with one being a write)
- (c) Start transaction
- (d) Rollback only

Answer: (b) — Standard conflict definition: both access same item and at least one is write.

14. Every conflict-serializable schedule is view-serializable, but the converse is false because of:

- (a) Commutative addition only
- (b) Blind writes
- (c) Force-writing the log
- (d) READ ONLY access mode

Answer: (b) — Blind writes allow view-serializable schedules that are not conflict-serializable.

15. In SQL, transaction initiation is typically:

- (a) Explicit **BEGIN TRANSACTION** required in all standards
- (b) Implicit when certain SQL statements are encountered
- (c) Automatic only after **COMMIT**
- (d) Never supported

Answer: (b) — Lecture: no explicit Begin in SQL2; initiation is implicit.

16. The **default** isolation level in SQL (as stated in the lecture) is:

- (a) READ UNCOMMITTED
- (b) READ COMMITTED
- (c) REPEATABLE READ
- (d) SERIALIZABLE

Answer: (d) — SQL2 default is SERIALIZABLE.

17. At isolation level **READ COMMITTED**, which anomaly is **prevented**?

- (a) Dirty read
- (b) Nonrepeatable read
- (c) Phantom
- (d) All three

Answer: (a) — Dirty read: no; nonrepeatable read and phantom: yes (still possible).

18. A **phantom** occurs when:

- (a) The log file disappears
- (b) A second execution of the same query returns rows that satisfy the condition but were inserted by another transaction
- (c) CPU cache is flushed
- (d) Two writes on the same item are lost

Answer: (b) — New rows appear that match T1's predicate on a repeat read.

19. **EXEC SQL ROLLBACK** in the sample embedded SQL transaction is used to:

- (a) Start a new transaction
- (b) Undo effects when an error path (**UNDO**) is taken
- (c) Force-write the log only
- (d) Set isolation to READ UNCOMMITTED

Answer: (b) — On **sqlerror**, control goes to UNDO and rolls back the transaction.

20. Concurrency control primarily addresses correctness under:

- (a) Interleaved execution
- (b) Tape backup only
- (c) Single-user batch mode only
- (d) Formatting disk blocks

Answer: (a) — Concurrency control handles interleaving; recovery handles failures.

SECTION B — STRUCTURED QUESTIONS (30 marks)

B1. Transaction states and recovery primitives (5 marks)

(a) List the five transaction states from the lecture. (2 marks)

(b) Name four recovery-manager operations (besides undo/redo). (3 marks)

Answer

(a) **States (2 marks — ½ mark each, any five):**

1. Active
2. Partially committed
3. Committed
4. Failed
5. Terminated

(b) **Operations (3 marks — 1 mark each, any four of):**

- **begin_transaction**
- **read / write** (on database items)
- **end_transaction**
- **commit_transaction**
- **rollback / abort**

(Undo and redo were excluded by the question.)

B2. Concurrency anomalies (5 marks)

For each scenario, name the anomaly: **lost update**, **dirty read (temporary update)**, or **incorrect summary**.

1. T1 writes X=100; T2 reads X=100; T1 aborts; T2 continues using 100.
2. T1 and T2 both read balance 500, add 50, write 550 — final balance 550 instead of 600.
3. T1 computes SUM(salary) while T2 updates some salaries mid-scan.

Answer

#	Anomaly	Brief reason
1	Dirty read (temporary update)	T2 read uncommitted data that may be rolled back.
2	Lost update	One update overwrote the other; effect of one transaction lost.
3	Incorrect summary	Aggregate mixed pre-update and post-update values.

(~1½ marks each for correct name + short justification.)

B3. Recoverability hierarchy (5 marks)

(a) Define **recoverable**, **cascadeless**, and **strict** schedules in one sentence each. (3 marks)

(b) State the implication relationship between the three. (2 marks)

Answer

(a)

- **Recoverable:** No transaction T commits until every transaction T' whose writes T has read has committed.
- **Cascadeless:** Every transaction reads only values written by **committed** transactions (no dirty reads).
- **Strict:** No transaction may read or write item X until the transaction that last wrote X has **committed**.

(b) **Strict** ⇒ **cascadeless** ⇒ **recoverable** (strict is strongest; strict schedules simplify recovery and avoid cascading rollbacks).

B4. Precedence graph (5 marks)

Schedule S:

r1(X); w1(X); r2(X); w2(X); c1; c2

(a) Draw the precedence graph (nodes T1, T2). (2 marks)

(b) Is S conflict-serializable? Give the equivalent serial order if yes. (3 marks)

Answer

(a) Conflicts: w1(X) before r2(X) → edge **T1** → **T2**. No edge T2 → T1 (no conflict with T2 before T1 on X in reverse). Graph: one directed edge T1 → T2.

(b) Graph is **acyclic** → **conflict-serializable**. Equivalent serial schedule: **T1, T2** (T1 before T2).

B5. SQL transaction and isolation (5 marks)

- (a) Write a minimal SQL sequence that (1) sets isolation to READ COMMITTED, (2) updates a table **Account**, (3) ends the transaction successfully. **(2 marks)**
- (b) Which two anomalies can still occur at READ COMMITTED? **(2 marks)**
- (c) Is a single **UPDATE** statement atomic in SQL? **(1 mark)**

Answer

(a) Example:

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
UPDATE Account SET balance = balance - 100 WHERE acc_id = 'A1';
COMMIT;
```

(Accept equivalent syntax; must include SET/COMMIT or implicit transaction rules as taught.)

(b) Nonrepeatable read and **phantom** (dirty read is blocked).

(c) Yes — a single SQL statement is atomic (completes fully or fails without partial effect on that statement).

B6. System log and recovery (5 marks)

- (a) List four types of log records from the lecture. **(2 marks)**
- (b) Explain how **undo** uses the log after a crash. **(1½ marks)**
- (c) Explain how **redo** uses the log after a crash. **(1½ marks)**

Answer

(a) Any four: `[start_transaction,T]`, `[read_item,T,X]`, `[write_item,T,X,old_value,new_value]`, `[commit,T]`, `[abort,T]`.

(b) Undo: Trace **backward** through log records for aborted/uncommitted transactions; reset each item changed by a write of T to **old_value**.

(c) Redo: Trace **forward** for **committed** transactions whose updates may not yet be on disk; reapply **new_value** for writes not permanently stored.

SECTION C — ESSAY QUESTIONS (50 marks)

C1. Why concurrency control and recovery are both needed (10 marks)

Discuss:

- Goals of transaction processing in a multiuser DBMS
- Three concurrency problems with examples
- Six failure causes from the lecture

- How logging supports recovery

Model answer

Goals: Modern DBMSs must be **correct** (consistent data, valid states) and **fast** (many users, interleaved or parallel work). Transactions bundle operations into atomic units.

Concurrency control prevents bad interleavings. **Lost update:** two transactions read-modify-write the same item; one update is lost. **Dirty read:** a transaction reads data written by another that later aborts, so the read was invalid. **Incorrect summary:** an aggregate runs while others update some rows, mixing old and new values in one result.

Recovery handles failures after or during execution: (1) system crash, (2) transaction/system errors, (3) local errors/abort conditions, (4) concurrency-control abort (serializability/deadlock), (5) disk failure, (6) catastrophes. Without recovery, partial updates violate **atomicity** and committed work may violate **durability**.

The **system log** on disk records starts, reads, writes (with old/new images), commits, and aborts. After a crash, **undo** rolls back uncommitted work using old values; **redo** reapplies committed work not yet on disk. **Force-writing** the log before commit ensures commit records and writes are durable enough for recovery.

(Award marks for: dual goals, three anomalies with clarity, several failure types, log + undo/redo + link to ACID.)

C2. ACID properties and commit point (10 marks)

Define each ACID property. Distinguish **consistency** from **isolation**. Explain the **commit point**, **force-write**, and why rollback is needed for transactions with `[start_transaction, T]` but no `[commit, T]`.

Model answer

- **Atomicity:** All operations of a transaction succeed together or none persist.
- **Consistency:** Execution takes the database from one **consistent** state (integrity constraints satisfied) to another; depends on constraints and correct transaction logic.
- **Isolation:** Concurrent transactions behave as if isolated; uncommitted updates should not affect others (strict isolation avoids dirty reads and cascading rollbacks).
- **Durability:** After commit, changes survive subsequent failures.

Consistency vs isolation: Consistency is about **valid database states** and constraints; isolation is about **scheduling** and visibility of concurrent operations.

Commit point: Reached when all DB operations of T have executed successfully and their effects are **recorded in the log**. After that, T is committed; `[commit, T]` is appended.

Force-write: Any log portion not yet on disk must be written to disk **before** commit point — so recovery can trust log contents after a crash (main memory may be lost).

Rollback: Transactions that started (log has start) but never committed must be **undone** so the database reflects neither partial nor erroneous effects.

C3. Recoverability vs serializability (10 marks)

Define **schedule**, **serial**, and **serializable** schedule. Compare **recoverable**, **cascadeless**, and **strict** schedules. Explain why recoverability and serializability are **different** correctness criteria, with a short example idea.

Model answer

A **schedule** (history) is an interleaving of operations from transactions $T_1 \dots T_n$ such that **within each T_i** , operations keep their original order.

A **serial** schedule runs each transaction's operations **consecutively** without interleaving other transactions' operations between them.

A **serializable** schedule is **equivalent** to some serial schedule (same net effect as some serial order) — it is a **correct** concurrent execution for consistency of data values.

Recoverability is about **commit dependencies**: if T_j reads a value written by T_i , T_j must not commit before T_i (recoverable). **Cascadeless**: only read **committed** data. **Strict**: no read/write X until last writer of X has committed. Chain: $\text{strict} \subset \text{cascadeless} \subset \text{recoverable}$ (generally).

Difference: A schedule can be **serializable** yet **not recoverable** (e.g., T_2 commits after reading uncommitted write from T_1 that later aborts — bad for recovery). Conversely, recoverability does not guarantee serializability. **Serializability** = equivalent to serial order for **correctness of final reads/writes**; **recoverability** = safe **commit/abort ordering** for recovery without cascading undo of committed transactions.

C4. Conflict vs view serializability and precedence graph (10 marks)

Define **conflict equivalence** and **view equivalence** (outline the three view conditions). Explain **blind writes** and why conflict-serializable $\subset$ view-serializable. Describe how to build and test a **precedence graph**. Mention one practical DBMS approach when full testing is hard.

Model answer

Conflict equivalent: Two schedules have the **same order** for every pair of **conflicting** operations (different transactions, same item, at least one write).

View equivalent (three conditions): (1) same transactions and operations; (2) each read in S reads from the **same writing transaction** (or initial value) as in S' ; (3) the **last write** to each item is the same transaction in both schedules.

Blind write: Write without reading the current value (e.g., $w_2(X)$ when T_2 never read X). Schedule $r_1(X); w_2(X); w_1(X); w_3(X)$ can be **view serializable** (equivalent to T_1, T_2, T_3) but **not conflict serializable** — no conflict equivalent serial schedule exists.

Precedence graph: Node per transaction; edge $T_i \rightarrow T_j$ if some operation in T_i **conflicts** and precedes an operation in T_j on the same item. **Acyclic** graph $\Leftrightarrow$ **conflict serializable**.

Practice: Checking entire schedules online is hard; DBMSs use **protocols** (e.g., **two-phase locking**) to ensure serializability without building full graphs at runtime. They often check **committed projection** of schedules.

C5. SQL transactions: syntax, isolation, and banking example (10 marks)

Explain implicit begin and explicit end in SQL. Describe `SET TRANSACTION` options (access mode, isolation, diagnostics). Complete the anomaly table for four isolation levels. Write a short **bank transfer** transaction in SQL (`SET TRANSACTION`, two updates, `COMMIT`/`ROLLBACK`) and state which anomalies `SERIALIZABLE` prevents.

Model answer

SQL transaction rules: Each **single SQL statement** is atomic. There is usually **no** explicit `BEGIN`; the transaction **starts implicitly**. It must end with `COMMIT` or `ROLLBACK`.

`SET TRANSACTION` can specify:

- **ACCESS MODE:** `READ ONLY` or `READ WRITE` (default `READ WRITE`; with `READ UNCOMMITTED`, `READ ONLY` may be assumed).
- **ISOLATION LEVEL:** `READ UNCOMMITTED`, `READ COMMITTED`, `REPEATABLE READ`, `SERIALIZABLE` (default `SERIALIZABLE`).
- **DIAGNOSTICS SIZE** `n`: number of conditions in diagnostic area.

Anomaly table:

Isolation level	Dirty read	Nonrepeatable read	Phantom
READ UNCOMMITTED	Yes	Yes	Yes
READ COMMITTED	No	Yes	Yes
REPEATABLE READ	No	No	Yes
SERIALIZABLE	No	No	No

Bank transfer example:

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
UPDATE Account SET balance = balance - 500 WHERE acc_id = 'FROM';
UPDATE Account SET balance = balance + 500 WHERE acc_id = 'TO';
-- Optional: check row counts / constraints
COMMIT;
-- On error handler: ROLLBACK;
```

If the first update succeeds and the second fails, `ROLLBACK` undoes both (transaction atomicity). At **`SERIALIZABLE`**, interleaving is equivalent to a serial order — **no dirty read, nonrepeatable read, or phantom** as defined in the lecture.

Lower levels trade **performance** for allowing more anomalies; choosing `READ COMMITTED` is common in practice when strict serializability is too costly.

MARKING GUIDE (INSTRUCTOR)

Section	Total	Notes
---------	-------	-------

Section	Total	Notes
A — MCQ	20	1 mark each; no partial credit
B1–B6	30	Partial credit for definitions, graphs, SQL syntax
C1–C5	50	10 marks each; rubric: definitions (3–4), examples (3–4), depth/linkage (2–3)

Suggested pass threshold: 50/100 (adjust per institution).

End of examination paper.